



Mediacrest Training College

Nairobi, Kenya

Advanced Data Analytics & Machine Learning

8-Week Advanced Training Program

Session Notes

Week 1 · Session 1

Advanced Data Auditing & Missingness Mechanisms

Course Trainer: Victor Wandera Lumumba

Role: Statistician, Data Analyst & BI Specialist

Session Date: 10 August 2026

Mode: Hybrid – Online (6–8 PM) / Physical (9–11 AM EAT)

“Audit before you model — every imputation is a modelling decision.”

About This Document

These notes accompany **Week 1, Session 1** of the **Advanced Data Analytics & Machine Learning** program. They are designed for analysts who have completed the Foundations of Data Analysis course (or equivalent) and now want to move from basic cleaning to *principled, policy-grade data preparation*.

Course at a Glance

	8 Weeks (20+ intensive sessions)
Duration	
Core Tools	Python (pandas, NumPy, statsmodels, scikit-learn, XGBoost), Power BI, AXI tools
Focus	Deep statistical modelling, supervised & unsupervised ML, diagnostics, explainability
Capstone	Production-ready ML system with diagnostics, explainability & policy recommendations
Kenyan Context	Healthcare, agriculture, finance, public policy (MOH, CBK, KRA, County Govts)

Where This Session Fits

Week 1 is titled “**Advanced Data Preparation, Data Visualization & Statistical Thinking**.” It is split into three sessions:

- 1. Session 1 (this document):** Advanced Data Auditing & Missingness Mechanisms.
- 2. Session 2:** Outlier detection, feature encoding, scaling & normalisation.
- 3. Session 3:** Data leakage prevention & reproducible preprocessing pipelines.

How to Use These Notes

- **Follow along in code.** Every listing is runnable. Open the companion notebook (*Cleaning the data like a pro*) and execute each cell as you read.
- **Boxes matter.** **Key Concept** boxes hold the theory you must internalise; **Warning** boxes hold the mistakes that fail models; **Kenyan Context** boxes connect the material to MOH/CBK/KRA/County work.
- **Do the exercises.** Each Part ends with practice tasks. The practical in Part 5 is graded.

Contents

About This Document	1
1 Session Overview & Learning Objectives	4
1.1 Session Objectives	4
1.2 Prerequisites	4
1.3 Key Terms for This Session	4
2 Part 1: Why Data Auditing Matters	5
2.1 The 80% Rule: Preparation Is the Work	5
2.2 Common Data-Quality Defects	5
2.3 The Statistical Framework for Missingness	6
2.4 MCAR — Missing Completely At Random	6
2.5 MAR — Missing At Random	6
2.6 MNAR — Missing Not At Random	7
2.7 Diagnosing the Mechanism in Practice	7
2.8 Treatment Decision Matrix	8
3 Part 2: Profiling & Exploratory Data Analysis	9
3.1 Step 1 – Install the Data-Profiling Library	9
3.2 Step 2 – Load the Required Libraries	9
3.3 Steps 3 & 4 – Load the Data and Generate the Profile	9
3.3.1 What to Look For in the Report	10
3.4 Step 5 – Exploratory Data Analysis	10
3.4.1 Check the Shape of the Data	10
3.4.2 Check the Data Types	10
3.4.3 Check the Number of Missing Values	10
3.4.4 Check the Percentage of Missing Values	11
3.5 Step 6 – Standardise the Column Names	11
3.6 Sort Columns by Missing Percentage	11
4 Part 3: Duplicates & Missing-Value Treatment	12
4.1 Step 7a – Detect Duplicate Rows	12
4.2 Step 7b — Inspect the Duplicated Rows	12
4.3 Step 7c — Drop the Duplicated Rows	12
4.4 Step 7d — Impute Numeric Columns with the Median	12
4.5 Step 7e — Impute Categorical Columns with “Unknown”	13
4.6 Step 7f — Verify the Result	13
5 Part 4: Standardising Values & the Extended Cleaning Toolkit	14
5.1 Standardise Categorical Values: Gender	14
5.2 Standardise Categorical Values: Region	14
5.3 Clean Whitespace: ProductCategory	14
5.4 Guard Continuous Variables Against Invalid Ranges	15
5.5 Treat Rating and Revenue Anomalies Consistently	15
5.6 Re-Profile the Cleaned Data	15
5.7 The Extended Toolkit: Real-World Messy Data	16

5.7.1	Load and Profile the Messy Data	16
5.7.2	Deep Text Cleaning: Employee Full Name	16
5.7.3	Empty Strings vs. Missing Values	16
5.7.4	Expanding Multi-Value Columns: Project Codes	17
5.7.5	Convert Date Columns to a Real Date Type	17
5.7.6	Cleaning a Currency Column: Base Salary	17
5.7.7	Cleaning a Stubborn Target: Bonus Target	18
5.7.8	Batch-Cleaning Several Money Columns	18
5.7.9	Validating Against a Fixed Category List	19
5.7.10	Final Sweep: Missingness, Duplicates, and a Derived Flag	19
5.7.11	Profile and Save the Final Dataset	20
6	Part 5: Practical Application — National Health Dataset	21
6.1	Scenario	21
6.2	Your Mission	21
6.3	Worked Solution Sketch	21
6.4	Bias-Identification Memo Template	22
7	Best-Practice Checklist	23
8	Common Pitfalls to Avoid	23
9	Glossary of Key Terms	24
10	Quick-Reference Cheat Sheet	25
11	References & Further Reading	25
12	Session Deliverables & What Comes Next	26

1 Session Overview & Learning Objectives

1.1 Session Objectives

By the end of this session you will be able to:

- Understand and classify the three **missingness mechanisms** — MCAR, MAR, MNAR — and match each to a safe treatment.
- Conduct an **advanced data audit**: automated profiling plus manual checks on shape, data types, and missingness.
- Standardise **column names** and **categorical values** to eliminate silent join and grouping errors.
- Apply principled **missing-value treatment**: median for numeric, “Unknown” for categorical.
- Identify **bias introduced by missing clinical records** in a Kenyan health dataset and document it for policy-grade reporting.

1.2 Prerequisites

- Basic Python and pandas (reading CSVs, `head()`, simple filtering).
- Jupyter Notebook or JupyterLab installed.
- The dataset `Business_Data_Quality_Issues.csv` (and later `Messy_data.csv`) in your working directory.

1.3 Key Terms for This Session

- | | |
|---------------------|------------------------|
| • Data audit | • Duplicate rows |
| • Data profiling | • Data type (dtype) |
| • Missingness | • Categorical encoding |
| • MCAR / MAR / MNAR | • Distribution skew |
| • Imputation | • Bias |

2 Part 1: Why Data Auditing Matters

2.1 The 80% Rule: Preparation Is the Work

A persistent myth in analytics is that the “interesting” work is the modelling. In practice, professional analysts spend roughly **80% of their time** collecting, cleaning, and preparing data, and only about 20% modelling and evaluating. This is not a failure of discipline — it is the nature of real-world data, which arrives with missing values, duplicates, inconsistent labels, wrong data types, and impossible numbers.

Key Concept

Every statistical and machine-learning model is only as good as the data it is trained on. A model cannot “see through” a bad dataset; it will faithfully learn and then reproduce whatever flaws, gaps, and biases the data contains. This is the principle of **“garbage in, garbage out.”**

In a policy setting the stakes are even higher. If you prepare a dataset for the Ministry of Health or a County Government and you silently mishandle missing clinical records, your model’s recommendations can misallocate scarce resources — for example, understating disease burden in the very facilities that need the most support.

2.2 Common Data-Quality Defects

Before you can fix problems you must be able to name them. Table 1 lists the defects you will encounter most often and their typical consequences.

Defect	What it looks like	Consequence
Missing values	NaN, blanks, NA, ?	Biased estimates, reduced power
Duplicate rows	Identical records repeated	Inflated weights, over-counting
Inconsistent labels	M/male/Male	Broken joins, phantom categories
Wrong data types	Dates/numbers stored as text	Silent NaN, failed math
Out-of-range values	Negative quantity, rating of 900	Skewed means, distorted models
Non-printable characters	Hidden control characters in text	Failed matches, dirty categories

Table 1: Common data-quality defects and their downstream consequences.

Kenyan Context

When merging tables from MOH, CBK, KRA, or a County Government, inconsistent labels are the single most common cause of failed joins. A county spelled “Nakuru” in one table and “NAKURU County” in another will produce two separate groups — silently splitting your analysis.

2.3 The Statistical Framework for Missingness

The most important idea in this session is that **not all missing data is the same**. The correct treatment depends entirely on *why* the data is missing. Statisticians distinguish three mechanisms, introduced by Rubin (1976).

mclight	Mechanism	Meaning	Detection clue	Safe treatment
MCAR		Missingness unrelated to anything observed or unobserved	Evenly spread; Little's test $p > 0.05$	Deletion or simple imputation
MAR		Missingness explained by <i>observed</i> variables	Missing rate varies by a known group	Conditional/model-based imputation
MNAR		Missingness depends on the <i>unobserved</i> value itself	Pattern tracks the value	Model the missingness; flag it

Table 2: The three missingness mechanisms at a glance.

2.4 MCAR — Missing Completely At Random

Definition. A value is Missing Completely At Random when the probability that it is missing is independent of everything — both the variables you have observed and the ones you have not. The missingness is pure noise.

Example. A laboratory machine jams at random one afternoon. Every sample that happened to be in the queue during that window is missing, regardless of the patient's health. Because the missingness has no relationship to any patient characteristic, deleting those rows does not bias your analysis.

Detection.

- Missingness appears evenly distributed across all groups.
- **Little's MCAR test:** a non-significant result ($p > 0.05$) is consistent with MCAR.
- Compare the distributions of the observed data between the “missing” and “present” groups — they should look the same.

Warning / Common Pitfall

True MCAR is **rare** in real datasets. Deletion is only truly safe under MCAR, so treat any convenient “just delete it” assumption with strong suspicion.

2.5 MAR — Missing At Random

Definition. A value is Missing At Random when its probability of being missing depends on *other observed variables*, but *not* on the missing value itself.

Example. Blood-pressure readings are missing more often in rural clinics that lack automated cuffs. The missingness is explained by *location* (which you have observed), not by the patient's actual blood pressure. You can impute conditionally on county or facility type.

Detection.

- The missing rate differs across known groups (county, gender, facility tier).

- You can model the missingness using observed predictors.

Treatment. Conditional imputation — group-wise medians, K-Nearest-Neighbours (KNN), or Multiple Imputation by Chained Equations (MICE).

The Name Is Misleading

“Missing At Random” does *not* mean the missingness is random. It is usually **systematic** — but the system is driven by variables you *can* observe. That is what makes it recoverable.

2.6 MNAR — Missing Not At Random

Definition. A value is Missing Not At Random when the probability of missingness depends on the *unobserved value itself*.

Example. Sicker patients are more likely to skip clinic visits, so their clinical records are missing *because* they are unwell. If you impute their vitals with the average of healthy patients, you systematically understate disease burden.

Detection.

- The hardest mechanism to prove statistically.
- Requires domain reasoning and careful pattern inspection.

Treatment. Model the missingness explicitly, run sensitivity analyses, and **always flag the risk** in your report.

Policy Danger

MNAR in a national health dataset can hide the very population a policy is meant to serve. If the sickest or most remote patients are the ones with missing records, a naively imputed model will under-prioritise the facilities that need the most investment.

2.7 Diagnosing the Mechanism in Practice

You can test for MCAR formally with Little’s test. The snippet below shows the idea (you will apply it in the practical):

```
1 # Little's MCAR test: H0 = data is MCAR.
2 # A large p-value (> 0.05) fails to reject MCAR.
3 # Many teams implement it manually or use a package.
4 # Always pair the test with group-wise missingness plots.
5 missing_rate = df.isnull().mean()
6
7 # Cross-tabulate missingness by a known group (e.g. county)
8 by_county = df.groupby("County")[["BloodPressure"]] \
9             .apply(lambda x: x.isnull().mean())
10 print(by_county.sort_values(ascending=False))
```

Listing 1: Conceptual outline of Little’s MCAR test

If the missing rate is roughly equal everywhere, suspect MCAR. If it varies strongly by a known group, suspect MAR. If you have a domain reason to believe the missingness tracks the value itself, treat it as MNAR and proceed with caution.

2.8 Treatment Decision Matrix

Use this decision rule every time you encounter a column with missing values:

Step 1: Quantify the missing share for the column.

Step 2: Classify the mechanism (MCAR / MAR / MNAR) using tests and group-wise inspection.

Step 3: Decide the treatment:

- MCAR + small share → deletion or simple imputation is acceptable.
- MAR → impute conditionally (group median, KNN, MICE).
- MNAR → model or flag; avoid naive mean/median.
- >70% missing → consider dropping the column as a feature.

Step 4: Document the decision and the assumption behind it.

Part 1 – Concept Check

- a. A hospital's electronic system was offline every Tuesday in 2024. Records from those Tuesdays are missing regardless of patient condition. Which mechanism is this, and is deletion safe?
- b. HIV test results are missing more often for patients who never returned for follow-up. Which mechanism, and what is the risk of median imputation here?
- c. Name two reasons why a column missing 80% of its values is usually a poor modelling feature.

3 Part 2: Profiling & Exploratory Data Analysis

In this part you build the **audit toolkit**: an automated profiling report plus a set of fast manual checks. The workflow mirrors the companion notebook exactly.

3.1 Step 1 – Install the Data-Profiling Library

A profiling library scans your DataFrame and produces an interactive HTML report — distributions, missingness, correlations, and automatic alerts — in a single command. Install one of the following (run once, in a notebook cell with the `!` prefix or in a terminal without it):

```
1 #!pip install fg-data-profiling
2 #!pip install ydata-profiling
```

Listing 2: Install a profiling package

Pro Tip

Prefer `ydata-profiling` if `fg-data-profiling` is unavailable in your environment. Both expose a similar `ProfileReport` API.

3.2 Step 2 – Load the Required Libraries

```
1 import pandas as pd
2 import webbrowser
3 import numpy as np
4 import data_profiling
5 from data_profiling import ProfileReport
6 #from fg_data_profiling import ProfileReport
```

Listing 3: Standard imports for the audit

Library	Purpose in this session
<code>pandas</code>	Load, inspect, and transform the data
<code>webbrowser</code>	Open the generated HTML report
<code>numpy</code>	Numeric operations and <code>np.nan</code>
<code>ProfileReport</code>	Generate the automated audit report

Table 3: Libraries and their roles.

ModuleNotFoundError

If Python reports No module named `'data_profiling'`, install the package (Step 1) and then **restart the kernel** before importing again. A running kernel caches failed imports.

3.3 Steps 3 & 4 – Load the Data and Generate the Profile

```
1 df = pd.read_csv("Business_Data_Quality_Issues.csv")
2 df.head()
3
4 profile = ProfileReport(df, title="Profiling Report",
5                         explorative=True)
```

```
6 profile.to_file('report.html')
7 webbrowser.open("report.html")
```

Listing 4: Load the raw data and profile it

The `explorative=True` flag runs a deeper (slower) analysis, including more detailed correlations. The report is written to `report.html` and opened in your browser.

3.3.1 What to Look For in the Report

- **Overview:** row/column counts, memory footprint, duplicate rate, and overall missingness.
- **Variables:** for each column — inferred type, missing %, distinct count, and distribution histogram.
- **Correlations:** highly correlated features (candidates for removal later in the course).
- **Alerts:** automatic flags for constant columns, highly-correlated pairs, skew, and zeros.

Key Concept

Generate this report **twice** during the session — once on the raw data and once after cleaning (Part 5). The difference between the two reports is your objective evidence of the work done.

3.4 Step 5 – Exploratory Data Analysis

Automated reports are convenient, but a professional always verifies with direct checks.

3.4.1 Check the Shape of the Data

```
df.shape
```

Listing 5: Row and column counts

`df.shape` returns a tuple (`rows`, `columns`). Record it now — you will compare it after every major operation.

3.4.2 Check the Data Types

```
df.dtypes
```

Listing 6: Data type of every column

Silent NaN from Wrong Types

If a column that *should* be numeric (e.g. `Revenue`) is typed as `object`, it usually contains currency symbols, commas, or text. Any numeric operation on it will silently fail or return NaN. Fix the type before any imputation.

3.4.3 Check the Number of Missing Values

```
df.isnull().sum()
```

Listing 7: Raw missing counts per column

3.4.4 Check the Percentage of Missing Values

```
df.isnull().mean()*100
```

Listing 8: Missing values as a percentage

Pro Tip

Always look at **percentages**, not just counts. A column with 5,000 missing values sounds alarming, but if the dataset has one million rows that is only 0.5% — usually harmless.

3.5 Step 6 – Standardise the Column Names

Column names with spaces, mixed case, and parentheses are a constant source of errors. Standardise them once, up front:

```
1 df.columns = (  
2     df.columns  
3     .str.strip()           # remove leading/trailing spaces  
4     .str.replace(' ', '_') # spaces -> underscores  
5     .str.replace('(', '')  # drop parentheses  
6     .str.replace(')', '')  
7 )  
8 df.head()
```

Listing 9: Clean and standardise column names

Why Standardise Names?

Uniform column names make filtering, selecting, merging, and model-building reliable. They remove the need to remember or retype awkward names with spaces or special characters. In the notebook the `.str.lower()` line is commented out; uncomment it if you want fully lowercase names.

3.6 Sort Columns by Missing Percentage

After standardising names, re-rank the columns by how much data they are missing, so you tackle the worst offenders first:

```
1 missing_values = df.isnull().mean() * 100  
2 missing_values = missing_values[missing_values > 0] \  
3     .sort_values(ascending=False)  
4 missing_values
```

Listing 10: Missing percentages, sorted worst-first

This gives you a prioritised to-do list for imputation.

Part 2 – Hands-On

- Load `Business_Data_Quality_Issues.csv` and record its `shape` and every column's `dtype`.
- Generate a `ProfileReport` and list three automatic alerts it raises.
- Identify the column with the highest missing percentage and classify its likely mechanism (MCAR/MAR/MNAR) with one sentence of reasoning.

4 Part 3: Duplicates & Missing-Value Treatment

4.1 Step 7a – Detect Duplicate Rows

Duplicates overweight certain records, distorting aggregates and any model trained on the data.

```
1 dupes = df.duplicated()
2 dupes.sum()
```

Listing 11: Count duplicate rows

4.2 Step 7b — Inspect the Duplicated Rows

Before deleting, look at the duplicates. `keep=False` marks *all* copies of a duplicated group, so you can see the full set and confirm they are true repeats rather than legitimate near-duplicates.

```
1 df[df.duplicated(keep=False)] \
2   .sort_values(by=list(df.columns)).head(10)
```

Listing 12: Print the duplicated rows for inspection

4.3 Step 7c — Drop the Duplicated Rows

```
1 df.drop_duplicates(keep='first', inplace=True)
2 df.shape
```

Listing 13: Remove duplicates, keeping the first occurrence

Key Concept

Compare `df.shape` before and after the drop. The reduction in row count should equal the duplicate count from Step 7a. If the dataset has a natural key (patient ID, transaction ID), deduplicate *on that key* rather than on all columns.

4.4 Step 7d — Impute Numeric Columns with the Median

Make a defensive copy, then fill every numeric column's missing values with its median:

```
1 df = df.copy()
2
3 numeric_columns = df.select_dtypes(include=np.number).columns
4 for col in numeric_columns:
5     df[col] = df[col].fillna(df[col].median())
```

Listing 14: Median imputation for numeric features

Why the Median?

The median is robust to outliers, so a few extreme values cannot drag the fill value. Reserve the **mean** for columns that are genuinely symmetric and free of outliers. In missingness terms, median imputation is a reasonable simple treatment when the share is small and the mechanism is closer to MCAR.

4.5 Step 7e — Impute Categorical Columns with “Unknown”

```
1 categorical_columns = df.select_dtypes(  
2     include=['object', 'category']).columns  
3 for col in categorical_columns:  
4     df[col] = df[col].fillna("Unknown")
```

Listing 15: Unknown imputation for categorical features

Why “Unknown” and Not the Mode?

Filling with the most frequent category injects false signal — it pretends you know a value you do not. “Unknown” preserves the *fact* of missingness as a legitimate category, which is often itself informative (for example, an undocumented facility tier). If records are missing *because* of the underlying condition (MNAR), “Unknown” keeps that signal visible for later investigation.

4.6 Step 7f — Verify the Result

```
1 print(df.isnull().sum())  
2 df.head()
```

Listing 16: Confirm no missing values remain

Empty Strings Are Not NaN

`isnull()` does **not** detect empty strings (“”). A cell can look “blank” yet not register as missing. This recurring trap is handled explicitly in the advanced toolkit (Part 4, Section 4.8).

Part 3 – Hands-On

- Count and then drop duplicates in your dataset. Report the row count before and after.
- Impute all numeric columns with the median and all categorical columns with “Unknown”. Verify that `isnull().sum()` is zero everywhere.
- Explain in one sentence why you chose the median over the mean for a right-skewed column like **Revenue**.

5 Part 4: Standardising Values & the Extended Cleaning Toolkit

Imputation fixes *missing* values; standardisation fixes *inconsistent* values. Both are required before a dataset is analysis-ready.

5.1 Standardise Categorical Values: Gender

```
1 df['Gender'].value_counts()
2
3 df["Gender"] = df["Gender"].replace({
4     "M": "Male",
5     "male": "Male",
6     "F": "Female",
7     "FEMALE": "Female"
8 })
9 print(df["Gender"].value_counts())
```

Listing 17: Inspect, then map Gender variants to canonical labels

Key Concept

M, male, and Male are three different categories to a model. A `replace` dictionary collapses them into two clean labels. Always re-run `value_counts()` afterwards to confirm the categories merged. This step is a prerequisite to the encoding work in Session 2.

5.2 Standardise Categorical Values: Region

```
1 df['Region'].value_counts()
2
3 df["Region"] = df["Region"].replace({
4     "NORTH": "North",
5     "SOUTH": "South",
6     "EAST": "East",
7     "WEST": "West",
8     "CENTRAL": "Central"
9 })
10 print(df["Region"].value_counts())
```

Listing 18: Normalise Region casing

Kenyan Context

Apply the same discipline to the **47 counties**. Maintain an official spelling list so that “Nakuru”, “NAKURU”, and “Nakuru County” never become three separate groups in a join or a groupby.

5.3 Clean Whitespace: ProductCategory

```
1 df["ProductCategory"] = (
2     df["ProductCategory"]
3     .str.strip()
4     .str.replace(r"\s+", " ", regex=True)
5 )
6 print(df["ProductCategory"].value_counts())
```

Listing 19: Strip and collapse whitespace

The Invisible Bug

"Electronics" and "Electronics " (with trailing spaces) are different strings. `.str.strip()` removes edge spaces and the `\s+` regex collapses repeated internal spaces into one.

5.4 Guard Continuous Variables Against Invalid Ranges

First, produce comprehensive summary statistics to spot distribution problems:

```
1 summary = df[['Quantity', 'UnitPrice', 'Discount',
2               'Rating', 'Revenue']].agg(
3     ['count', 'mean', 'median', 'std', 'var', 'min',
4     'max', 'skew', 'kurt']).T
5 summary
```

Listing 20: Summary statistics for the continuous columns

`skew` and `kurt` flag distribution shape problems; `min/max` reveal impossible values. Then convert impossibilities to `NaN` *before* imputing — never impute over garbage:

```
1 df.loc[df["Quantity"] < 0, "Quantity"] = np.nan
2 df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median())
3 print(df["Quantity"].value_counts().sort_index())
```

Listing 21: Fix negative Quantity, then impute

5.5 Treat Rating and Revenue Anomalies Consistently

```
1 # Negative ratings -> NaN -> median
2 df.loc[df["Rating"] < 0, "Rating"] = np.nan
3 df["Rating"] = df["Rating"].fillna(df["Rating"].median())
4
5 # Negative revenue -> NaN -> median
6 df.loc[df["Revenue"] < 0, "Revenue"] = np.nan
7 df["Revenue"] = df["Revenue"].fillna(df["Revenue"].median())
```

Listing 22: Fix out-of-range Rating and Revenue

A Reusable Three-Step Pattern

For every bounded column (Rating 0–5, Revenue ≥ 0 , vitals within physiological range) use the same pattern:

1. **Detect** the impossible values.
2. **Nullify** them (`np.nan`).
3. **Impute** with the median.

5.6 Re-Profile the Cleaned Data


```

1 profile = ProfileReport(df, title="Profiling Report",
2                           explorative=True)
3 profile.to_file('report2.html')
4 webbrowser.open("report2.html")

```

Listing 23: Profile again to prove the improvement

5.7 The Extended Toolkit: Real-World Messy Data

The second half of the notebook (Messy_data.csv) demonstrates the techniques you will need on truly messy production data. These extend the same principles.

5.7.1 Load and Profile the Messy Data

```

1 df = pd.read_csv("Messy_data.csv")
2 df.head(20)
3
4 profile = ProfileReport(df, title="Profiling Report",
5                           explorative=True)
6 profile.to_file('report2.html')
7 webbrowser.open("report2.html")

```

Listing 24: Load Messy_data.csv and profile it

5.7.2 Deep Text Cleaning: Employee Full Name

This single block combines four classic operations: removing non-printable characters (CLEAN), trimming (TRIM), collapsing spaces, and proper-casing (PROPER).

```

1 import re
2 df["Employee_Full_Name"] = (
3     df["Employee_Full_Name"]
4     .str.replace(r"[^\x20-\x7E]", "", regex=True) # CLEAN: non-printable
5     .str.strip() # TRIM edges
6     .str.replace(r"\s+", " ", regex=True) # collapse spaces
7     .str.title() # PROPER case
8 )
9 df.head()

```

Listing 25: Clean, trim, and proper-case a name column

5.7.3 Empty Strings vs. Missing Values

This is the trap flagged in Part 3. Count each separately, then count them together:

```

1 empty_strings = (df["Employee_ID"] == "").sum()
2 print("Empty text strings:", empty_strings)
3
4 missing_values = df["Employee_ID"].isna().sum()
5 print("Missing values:", missing_values)
6
7 # Combined total
8 total_missing = ((df["Employee_ID"] == "") |
9                  (df["Employee_ID"].isna())) .sum()
10 print("Total empty strings and missing values:", total_missing)

```

Listing 26: Distinguish empty strings from NaN

5.7.4 Expanding Multi-Value Columns: Project Codes

A single column can hold several values separated by semicolons. Standardise it, then explode it into one binary (0/1) column per valid project — a manual form of one-hot encoding.

```
1 df["Project_Codes"] = (  
2     df["Project_Codes"]  
3     .fillna("")  
4     .str.strip()  
5     .str.replace(r"\s*;\s*", ";", regex=True)  
6     .str.title()  
7 )  
8  
9 projects = ["Alpha", "Beta", "Gamma",  
10            "Delta", "Epsilon", "Zeta"]  
11  
12 for project in projects:  
13     df[project] = df["Project_Codes"].str.contains(  
14         rf"\b{project}\b", case=False, regex=True  
15     ).astype(int)  
16 df.head()
```

Listing 27: Standardise then expand project codes

5.7.5 Convert Date Columns to a Real Date Type

```
1 date_columns = ["Hire_Date", "Invoice_Date",  
2                "Payment_Due_Date", "Payment_Received_Date"]  
3  
4 for col in date_columns:  
5     df[col] = pd.to_datetime(df[col], errors="coerce")  
6 df.head()  
7  
8 df[['Hire_Date', 'Invoice_Date', 'Payment_Due_Date',  
9     'Payment_Received_Date']].info()
```

Listing 28: Coerce several date columns at once

Key Concept

`errors="coerce"` turns unparseable values into NaT (Not a Time) instead of raising an error. After conversion, check both `isnull().sum()` and empty-string counts on the date columns.

5.7.6 Cleaning a Currency Column: Base Salary

Currency columns arrive with symbols, commas, and currency codes. Strip them, coerce to numeric, then treat non-positive salaries as missing before imputing:

```
1 df["Base_Salary"] = (  
2     df["Base_Salary"]  
3     .astype(str)  
4     .str.replace(r"[$,]", "", regex=True)    # symbols + commas
```

```

5     .str.replace("KSh", "", regex=False)    # remove KSh
6     .str.strip()
7 )
8 df["Base_Salary"] = pd.to_numeric(df["Base_Salary"],
9                                   errors="coerce")
10 df.loc[df["Base_Salary"] <= 0, "Base_Salary"] = np.nan
11 df["Base_Salary"] = df["Base_Salary"].fillna(
12     df["Base_Salary"].median())
13 df["Base_Salary"].describe()

```

Listing 29: Clean *BaseSalary*

Kenyan Context

The KSh replacement is essential for Kenyan payroll and budget data. Extend the regex to include £ and € if your source mixes currencies.

5.7.7 Cleaning a Stubborn Target: Bonus Target

Some columns contain literal text like NA, na, or "nan" that must be converted to real missing values before numeric coercion:

```

1 df["Bonus_Target"] = (
2     df["Bonus_Target"]
3     .replace(r'\s*$', np.nan, regex=True) # spaces -> NaN
4     .replace("NA", np.nan).replace("na", np.nan)
5     .astype(str)
6     .str.replace(r"[$,]", "", regex=True)
7     .str.replace("KSh", "", regex=False)
8     .str.strip()
9     .replace({"nan": np.nan})             # "nan" -> NaN
10 )
11 df["Bonus_Target"] = pd.to_numeric(df["Bonus_Target"],
12                                   errors="coerce")
13 df["Bonus_Target"] = df["Bonus_Target"].fillna(
14     df["Bonus_Target"].median())
15
16 print(df["Bonus_Target"].describe())
17 print("Missing values:", df["Bonus_Target"].isna().sum())

```

Listing 30: Clean *BonusTarget*

5.7.8 Batch-Cleaning Several Money Columns

When several columns need identical treatment, loop over them:

```

1 columns = ["Actual_Revenue_Generated", "Project_Budget",
2            "Actual_Expenses"]
3 for col in columns:
4     df[col] = (
5         df[col]
6         .replace(r'\s*$', np.nan, regex=True)
7         .replace(["NA", "na", "N/A", "n/a"], np.nan)
8         .astype(str)
9         .str.replace(r"[$,]", "", regex=True)
10        .str.replace("KSh", "", regex=False)
11        .str.strip()

```

```

12         .replace({"nan": np.nan})
13     )
14     df[col] = pd.to_numeric(df[col], errors="coerce")
15     df.loc[df[col] < 0, col] = np.nan
16     df[col] = df[col].fillna(df[col].median())
17
18     print(df[columns].describe())
19     print(df[columns].isna().sum())

```

Listing 31: Clean revenue, budget, and expense columns together

5.7.9 Validating Against a Fixed Category List

After cleaning a categorical column, replace any value outside an approved list with “Unknown”. This guarantees a stable, known set of categories for encoding:

```

1 df["Status_Flag"] = (
2     df["Status_Flag"]
3     .astype(str)
4     .str.replace(r"[\x00-\x1F\x7F-\x9F]", "", regex=True)
5     .str.replace(r"\s+", " ", regex=True)
6     .str.strip()
7     .str.title()
8     .replace({"Nan": np.nan, " ": np.nan})
9 )
10 df["Status_Flag"] = df["Status_Flag"].fillna("Unknown")
11
12 valid_status = ["Active", "Inactive", "Pending",
13                "On Leave", "Unknown"]
14 df.loc[~df["Status_Flag"].isin(valid_status),
15        "Status_Flag"] = "Unknown"
16 print(df["Status_Flag"].value_counts())

```

Listing 32: Clean Status_{Flag} against a valid list

```

1 df["Region_Code"] = (
2     df["Region_Code"]
3     .replace(r'^\s*$', np.nan, regex=True)
4     .replace(["NaN", "nan", "NA", "N/A"], np.nan)
5     .astype(str)
6     .str.replace(r"[\x00-\x1F\x7F-\x9F]", "", regex=True)
7     .str.strip()
8     .str.upper()
9     .replace({"NAN": np.nan})
10 )
11 df["Region_Code"] = df["Region_Code"].fillna("Unknown")
12
13 valid_regions = ["EU", "APAC", "LATAM", "Unknown"]
14 df.loc[~df["Region_Code"].isin(valid_regions),
15        "Region_Code"] = "Unknown"
16 print(df["Region_Code"].value_counts())

```

Listing 33: Clean Region_{Code} against a valid list

5.7.10 Final Sweep: Missingness, Duplicates, and a Derived Flag

```
1 df.isnull().sum()           # overall missingness
2 df.duplicated().sum()      # remaining duplicates
3
4 df[df.duplicated(keep=False)] \
5     .sort_values(by=list(df.columns)).head(10)
6 df = df.drop_duplicates()
7 df.shape
8 df.info()
9
10 # Derive a Payment_Status flag from a date column
11 df["Payment_Status"] = np.where(
12     df["Payment_Received_Date"].isna(),
13     "Unpaid", "Paid")
14 df.head(20)
```

Listing 34: Final checks and a derived *Payment_{status}flag*

Derive, Don't Just Clean

Payment_Status is a *derived feature* created from the cleaned date column. Feature derivation like this often improves models and is explored further in later weeks.

5.7.11 Profile and Save the Final Dataset

```
1 profile = ProfileReport(df, title="Profiling Report",
2                         explorative=True)
3 profile.to_file('report3.html')
4 webbrowser.open("report3.html")
5
6 df.to_csv("Cleaned_Business_Data.csv", index=False)
```

Listing 35: Final profile and export

Warning / Common Pitfall

Always export with `index=False` unless you specifically need the index. Otherwise you write an extra, unnamed column that causes confusion when the file is reloaded.

6 Part 5: Practical Application — National Health Dataset

6.1 Scenario

The Ministry of Health hands you a national patient dataset to build a predictive model for **hypertension risk**. Clinical records are unevenly complete across counties: well-resourced urban facilities submit full records, while some rural facilities have sparse vitals and missing lab results.

6.2 Your Mission

Apply today's toolkit end-to-end:

1. **Profile** the dataset with `ProfileReport` and record every quality alert.
2. **Quantify** missingness per column and **classify** the mechanism (MCAR/MAR/MNAR) for the three worst columns.
3. **Standardise** facility names, the 47 county labels, and categorical codes.
4. **Treat** missing values: median for vitals, "Unknown" for facility/cadre categories.
5. **Document** the bias introduced by missing clinical records in a short memo.

6.3 Worked Solution Sketch

The code below shows the skeleton. Substitute your real column names.

```
1 import pandas as pd
2 import numpy as np
3
4 df = pd.read_csv("moh_national_health.csv")
5
6 # 1. Profile (run in notebook) and record alerts
7 # profile = ProfileReport(df, explorative=True)
8
9 # 2. Quantify + classify missingness
10 missing = (df.isnull().mean()*100).sort_values(ascending=False)
11 print(missing.head(10))
12
13 # Cross-tabulate the worst column by county
14 by_county = df.groupby("County")["BloodPressure"] \
15             .apply(lambda x: x.isnull().mean())
16 print(by_county.sort_values(ascending=False).head(10))
17
18 # 3. Standardise county labels (example list)
19 df["County"] = df["County"].str.strip().str.title()
20
21 # 4. Treat missing values
22 num_cols = df.select_dtypes(include=np.number).columns
23 for c in num_cols:
24     df[c] = df[c].fillna(df[c].median())
25
26 cat_cols = df.select_dtypes(include=['object', 'category']).columns
27 for c in cat_cols:
28     df[c] = df[c].fillna("Unknown")
29
30 # 5. Verify
```

```
31 print(df.isnull().sum().sum()) # should be 0
```

Listing 36: Practical skeleton — audit to imputation

6.4 Bias-Identification Memo Template

Your deliverable is a short memo. Use this structure:

1. **Columns affected:** name each column and its missing share.
2. **Likely mechanism:** MCAR / MAR / MNAR, with one sentence of evidence.
3. **Treatment applied:** median, “Unknown”, or other, and why.
4. **Residual risk:** what bias could remain despite the treatment.
5. **Recommendation:** e.g. impute conditionally on county, or flag for the modelling team.

Expected Finding

If missing vitals cluster in under-resourced counties, naive median imputation pulls those counties’ averages toward the national mean — **understating** their disease burden. A model trained on that data will then under-prioritise the very facilities that need support. Your memo must say this explicitly.

7 Best-Practice Checklist

Use this checklist to close every data-preparation session:

- **Profile twice.** Once before cleaning, once after — the diff is your evidence of work.
- **Classify before you impute.** MCAR/MAR/MNAR determines whether deletion, median, or model-based imputation is safe.
- **Convert, then impute.** Turn impossible or non-numeric values into NaN *before* filling.
- **Standardise relentlessly.** Column names and categorical labels must be canonical before any join or encoding.
- **Watch for empty strings.** `isnull()` misses `""`; always check both.
- **Deduplicate on a key.** If a natural key exists, deduplicate on it, not on all columns.
- **Validate categories.** Replace unexpected values with “Unknown” against an approved list.
- **Document every decision.** What was dropped, what was filled, and why — this is your audit trail for MOH, CBK, or county reviewers.

8 Common Pitfalls to Avoid

Pitfall	Why it hurts / What to do instead
Deleting rows without checking the mechanism	Safe only under MCAR; classify first.
Mean-imputing a skewed column	Outliers drag the mean; prefer the median.
Mode-imputing categories	Injects false signal; use “Unknown”.
Imputing over impossible values	Nullify negatives/out-of-range first.
Averaging daily percentages	Divide totals instead (weighted metric).
Ignoring empty strings	They evade <code>isnull()</code> ; count them explicitly.
Exporting with the index	Adds a spurious column; use <code>index=False</code> .

Table 4: Recurring data-cleaning mistakes and their remedies.

9 Glossary of Key Terms

Data audit	A systematic inspection of a dataset for quality defects before analysis or modelling.
Data profiling	Automated generation of summary statistics, distributions, and alerts for a dataset.
MCAR	Missing Completely At Random; missingness unrelated to any variable. Deletion is safe.
MAR	Missing At Random; missingness explained by observed variables. Impute conditionally.
MNAR	Missing Not At Random; missingness depends on the unobserved value. Model or flag it.
Little's MCAR test	A statistical test whose null hypothesis is that data is MCAR.
Imputation	Replacing missing values with substituted values (median, mode, model estimate).
Median imputation	Filling missing numeric values with the column median; robust to outliers.
Duplicate rows	Identical records appearing more than once; they overweight observations.
dtype	The data type pandas assigns to a column (int, float, object, datetime).
Coercion	Forcing values to a target type, sending failures to NaN/NaT.
Empty string	A text value "" that is <i>not</i> detected by <code>isnull()</code> .
One-hot encoding	Creating one binary column per category (see Project Codes example).
Derived feature	A new column computed from existing columns (e.g. <code>Payment_Status</code>).
Skew	Asymmetry of a distribution; informs median vs. mean choice.
Bias	Systematic error in data or a model that distorts conclusions.
Analysis-ready	A dataset that is clean, consistently typed, and safe to model.

10 Quick-Reference Cheat Sheet

Task	Code
Shape of data	<code>df.shape</code>
Data types	<code>df.dtypes</code>
Missing counts	<code>df.isnull().sum()</code>
Missing %	<code>df.isnull().mean()*100</code>
Duplicate count	<code>df.duplicated().sum()</code>
Drop duplicates	<code>df.drop_duplicates(keep='first')</code>
Median imputation	<code>df[c].fillna(df[c].median())</code>
Unknown imputation	<code>df[c].fillna("Unknown")</code>
Standardise names	<code>df.columns.str.strip().str.replace(' ', '_')</code>
Map categories	<code>df[c].replace({...})</code>
Strip whitespace	<code>df[c].str.strip()</code>
Nullify negatives	<code>df.loc[df[c]<0, c]=np.nan</code>
Coerce to date	<code>pd.to_datetime(df[c], errors="coerce")</code>
Coerce to number	<code>pd.to_numeric(df[c], errors="coerce")</code>
Empty-string count	<code>(df[c]=="").sum()</code>
Save clean data	<code>df.to_csv("out.csv", index=False)</code>

11 References & Further Reading

- Rubin, D. B. (1976). *Inference and Missing Data*. Biometrika, 63(3), 581–592.
- Little, R. J. A. (1988). *A Test of Missing Completely at Random*. JASA, 83(404).
- pandas documentation — `isnull()`, `fillna()`, `replace()`, `to_datetime()`.
- ydata-profiling documentation for automated data profiling.
- Mediacrest Training College — *Advanced Data Analytics & Machine Learning* curriculum (Weeks 1–8).

12 Session Deliverables & What Comes Next

Today's Deliverable — due end of session

- Completed **data audit report** on the health dataset (before/after profiling).
 - Documented **missingness classification** and chosen treatment strategy per column.
 - A one-page **bias memo** on the missing clinical records.
- **Session 2:** Outlier detection (IQR, Z-score, influence), feature encoding, scaling & normalisation.
 - **Session 3:** Data leakage prevention and reproducible **sklearn** preprocessing pipelines.

Mediacrest Training College – Nairobi, Kenya

Advanced Data Analytics & Machine Learning · Week 1, Session 1

Trainer: Victor Wandera Lumumba · 10 August 2026
